

Lista-5 - Variáveis de Referência, Construtores, Sobrecarga

Informações do Autor

- **Nome:** [Roussian Gaioso](#)
 - **Data de atualização:** 28/09/2026
-

Regras de Entrega

- Grupos no máximo de 3 integrantes;
 - Trabalhos iguais de grupos diferentes, nota zero;
 - **Qualquer tentativa de fraude implicará em nota 0,0 (zero) na atividade, sem prejuízo de outras sanções;**
 - O grupo é responsável pela submissão, assim arquivos corrompidos ou ausência de arquivos impactará a nota ou a zerará;
 - Os nomes dos integrantes devem estar no arquivo [readme.md](#) dentro da submissão;
 - Caso a submissão seja reaberta, os trabalhos submetidos após a data prevista terão suas notas reduzidas pela metade;
 - Esse trabalho contará um percentual da **Nota de Atividades**, a ser definido no final do bimestre.
-

Questão-1

Um sistema de entregas precisa representar clientes e seus respectivos endereços.

Inicialmente foram criadas as seguintes classes:

```

public class Endereco {

    private String rua;
    private int numero;
    private String cidade;

    public Endereco(
        String rua,
        int numero,
        String cidade) {

        this.rua = rua;
        this.numero = numero;
        this.cidade = cidade;
    }
}

```

```

public class Cliente {

    private String nome;
    private String cpf;

    public Cliente(
        String nome,
        String cpf) {

        this.nome = nome;
        this.cpf = cpf;
    }
}

```

A equipe precisa agora relacionar um cliente ao seu endereço.

Um programador sugeriu copiar os dados do endereço para Cliente:

```

public class Cliente {

    private String nome;
    private String cpf;

    private String rua;
    private int numero;
    private String cidade;

}

```

Outro programador sugeriu:

```
public class Cliente {  
  
    private String nome;  
    private String cpf;  
  
    private Endereco endereco;  
  
}
```

Considere que cada cliente possui um endereço principal e que o sistema, futuramente, também precisará representar fornecedores e funcionários com endereço.

1. Escolha uma das duas formas de modelagem e justifique sua decisão, considerando os cenários descritos nos itens 8 e 9 desta questão.
2. Caso utilize:

```
private Endereco endereco;
```

explique por que endereco é uma variável do tipo referência.

3. Explique o relacionamento existente na seguinte situação:

```
private Endereco endereco;
```

utilizando o conceito de relacionamento TEM-UM.

4. Implemente um construtor para Cliente que receba:

```
nome  
CPF  
endereço
```

Implemente também, na classe Endereco, os métodos de acesso necessários para que os dados do endereço possam ser exibidos.

5. Na classe principal, crie primeiro um objeto Endereco:

```
Endereco endereco =  
    new Endereco(  
        "Rua das Flores",  
        100,  
        "Goiânia"  
    );
```

Depois crie um `Cliente` que possua esse endereço e exiba na tela o nome do cliente e os dados do seu endereço.

6. Explique o que é armazenado conceitualmente na variável:

```
Endereco endereco
```

quando um objeto é atribuído a ela.

7. Considere agora:

```
Endereco endereco = null;
```

Explique o significado de `null` nesse caso.

8. Qual seria uma desvantagem de copiar rua, número e cidade para várias classes diferentes, como `Cliente`, `Fornecedor` e `Funcionario`, em vez de reutilizar a classe `Endereco`?

9. Se futuramente a empresa adicionar o atributo `cep` ao endereço, compare o impacto dessa alteração nas duas propostas de modelagem.

Questão-2

Um sistema de cinema possui a seguinte classe:

```
public class Ingresso {  
  
    private String filme;  
    private double valor;  
  
    public Ingresso(  
        String filme,  
        double valor) {  
  
        this.filme = filme;  
        this.valor = valor;  
    }  
  
    public double calcularValor() {  
        return valor;  
    }  
}
```

Depois de algum tempo, o cinema passou a oferecer diferentes formas de calcular o valor do ingresso.

Para clientes comuns:

valor normal

Para estudantes:

valor normal - percentual de desconto

Para promoções especiais:

valor normal - valor fixo de desconto

A equipe deseja manter o método existente e adicionar novas formas de executar o cálculo.

Considere a seguinte proposta:

```

public double calcularValor() {
    return valor;
}

public double calcularValor(
    double percentualDesconto) {

    return valor -
        (valor * percentualDesconto / 100);
}

public double calcularValor(
    double desconto,
    boolean descontoFixo) {

    if (descontoFixo) {
        return valor - desconto;
    }

    return valor;
}

```

1. Identifique quais métodos estão sobrecarregados.
2. Explique por que Java consegue diferenciar os três métodos.
3. A seguinte implementação seria válida?

```

public double calcularValor(double desconto) {
    return valor - desconto;
}

public int calcularValor(double desconto) {
    return (int) (valor - desconto);
}

```

Explique sua resposta.

4. Considere:

```

public double calcularValor(double percentual) {
    ...
}

public double calcularValor(double desconto) {
    ...
}

```

Explique por que alterar somente o nome do parâmetro não cria uma sobrecarga válida.

5. Observe o terceiro método da proposta apresentada. Quando `descontoFixo` recebe `false`, o parâmetro `desconto` é ignorado e o retorno é igual ao de `calcularValor()`. Critique essa decisão de projeto: o código compila, mas o que ele comunica a quem o utiliza?
6. Implemente uma versão da classe `Ingresso` que permita calcular:

```
valor sem desconto  
valor com desconto percentual  
valor com desconto fixo
```

Observe que o desconto percentual e o desconto fixo são representados pelo mesmo tipo, o que impede diferenciá-los apenas pela lista de parâmetros. Resolva essa limitação e explique a estratégia escolhida, deixando claro por que a sobrecarga, isoladamente, não é suficiente nesse caso.

7. Na classe principal, demonstre as três chamadas.
 8. Qual é a vantagem de utilizar sobrecarga nesse caso?
 9. Qual pode ser a desvantagem de criar muitas versões sobrecarregadas de um mesmo método?
 10. Explique por que nem toda situação em que métodos realizam operações parecidas precisa necessariamente ser resolvida com sobrecarga.
-